

基础优化速查

```
import sys
sys.setrecursionlimit(600000) # 设置递归限制
```

TLE?

加算法

MLE?

检查重复定义

去除切片

递归转迭代

内置问题速查

内置删除

```
list
l.remove('指定元素')

cnt={}
del cnt[out]
```

gcd

```
from math import gcd
gcd(a,b)#最大公因数
#最小公倍数*最大公因数=a*b
```

permutations

```
from itertools import permutations

for i in itertools.permutations([1, 2, 3]):
    print(i, end=" ")
#(1, 2, 3) (1, 3, 2) (2, 1, 3) (2, 3, 1) (3, 1, 2) (3, 2, 1)

for i in itertools.permutations([1, 2, 3], 2):
    print(i, end=" ")
    print(type(i))
#(1, 2) <class 'tuple'>
#(1, 3) <class 'tuple'>
#(2, 1) <class 'tuple'>
#(2, 3) <class 'tuple'>
#(3, 1) <class 'tuple'>
```

```
#(3, 2) <class 'tuple'>
```

combinations

```
from itertools import combinations
combinations(list,k)      #生成list的k元组合（无序）（每个以元组形式存在）
```

math

```
math.pow(x, y) == x**y
math.factorial(n) == n!
```

输入

```
data = sys.stdin.read().split()
```

输出

保留小数

```
f = 2.3456789

print('%.2f'%f)
print('%.3f'%f)
print('%.4f'%f)

#2.35
#2.346
#2.3457
print(round(cur))#保留整数
```

大小写转换

```
a.upper()
a.lower()
```

f' '

```
f"{s:.2f}"#保留两位小数
```

join

```
print(' '.join(['1','2','3']))
print(' '.join(map(str,p)))
#'\n' 换行
```

基础数据结构

set

```
p=set()
#集合，快速查找的无重复元素集
p.add(1)
in p #快速判断
p.remove(元素名)
p.discard(元素名)#不会报错
a|b #并
a&b #交
a-b #表示∈a且∉b的元素的集合
a^b #等于a|b-a&b
a>b a<=b #比较从属关系，若无则为False
```

defaultdit

```
from collections import defaultdict
#这是一种"如果访问的key不存在与字典中,就给它新建一个默认值 dict[key]=default_value 的字典
vd=defaultdict(int) #以0为默认值的defaultdict
a=defaultdict(list) #以空列表为默认值的defaultdict
#类似地,括号里是int,set也可以
```

deque

```
import deque
```

heapq

```
import heapq
a = [] #创建一个空堆
heapq.heappush(a,18)

heapq.heappop(h) #输出最小值
```

排序

merge sort

```
def merge(left, right):
    result = []
    i = j = 0
    # 比较两个数组的元素，将较小的元素加入结果
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    # 将剩余元素加入结果
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort_iterative(arr):
    if len(arr) <= 1:
        return arr

    step = 1 # 初始子数组大小
    n = len(arr)

    while step < n:
        # 遍历所有子数组，每次处理两个相邻的子数组
        for i in range(0, n, step * 2):
            # 计算右边界，防止越界
            right_bound = min(i + step * 2, n)
            # 合并 [i, i+step) 和 [i+step, right_bound) 两个子数组
            left_arr = arr[i:i + step]
            right_arr = arr[i + step:right_bound]
            merged = merge(left_arr, right_arr)
            # 将合并结果写回原数组
            arr[i:right_bound] = merged
        step *= 2 # 子数组大小加倍
    return arr

# 测试示例
if __name__ == "__main__":
    arr = [5, 2, 6, 0, 3, 8, 4, 7, 1]
    sorted_arr = merge_sort_iterative(arr)
    print("排序前:", [5, 2, 6, 0, 3, 8, 4, 7, 1])
    print("排序后:", sorted_arr)
```

二分查找

区间左闭右闭, while left <= right , , if nums[middle] > target , right=middle - 1 , else: left = middle + 1

bisect

```
import bisect

# 已排序的列表
a = [1, 2, 4, 7]

# 查找元素 3 应该插入的位置
index = bisect.bisect_left(a, 3)
print(index) # 输出: 2

# 插入元素 3 到列表 a 中
bisect.insort_left(a, 3)
print(a) # 输出: [1, 2, 3, 4, 7]

#自己的代码
l=max(it)
r=sum(it)+1
k=0
while l<r : #1
    p=(l+r)//2
    s=1
    su=0
    for i in it:
        if su+i>p:
            su=i
            s+=1
        else:
            su+=i
    if s>M: #2
        l=p+1
    else:
        r=p
print(l) #3
```

数组

prefix——前序和

KMP

```
def compute_next(s):
    n = len(s)
    next_arr = [0] * (n + 1)
    next_arr[0] = -1
    j = 0
    for i in range(1, n):
        while j >= 0 and s[i] != s[j]:
            j = next_arr[j]
        j += 1
        next_arr[i + 1] = j
    return next_arr
```

next_arr[i] 为 前i个字符串的公共前后缀

monotonic stack——单调栈

接雨水，柱状图找最大矩形

并查集

Disjoint set(并查集)(Union find)

Union find 操作，find 时顺便简化指针

链表

判断环

快慢遍历至重合，第三指针从起点再开始与慢指针相遇

树

完全二叉树：层序无空点

中序转后序

```
for _ in range(int(input())):
    l=input()
    ans=[]
    s=[]
    i=-1
    pro={'*':2, '/':2, '+':1, '-':1, '(':0}
```

```

while i<len(l)-1:
    i+=1
    it=l[i]
    if it==' ':
        continue
    if ord('0')<=ord(it)<=ord('9'):
        p=i
        while i<len(l) and (ord('0')<=ord(l[i])<=ord('9') or l[i]=='.'):
            i+=1
        ans.append(l[p:i])
        i-=1
        continue
    if it=='(':
        s.append(it)
        continue
    if it==')':
        while s[-1]!='(':
            ans.append(s.pop())
            s.pop()
        continue
    while s and s[-1]!='(' and pro[s[-1]]>=pro[it]:
        ans.append(s.pop())
    s.append(it)
while s:
    ans.append(s.pop())
print(' '.join(ans))

```

普里姆(Prim)算法和克鲁斯卡尔(Kruskal)算法——最小生成树(MST)

prim---归并顶点；kruskal--归并边(sort权值先)

trie树——基于前缀存储

Huffman

在F中选取两棵根节点权值最小的作为树的左右孩子构造一棵新的二叉树，且置根节点的权值为左右孩子权值之和，在F中删除这两棵树，新二叉树之于F中

手搓堆(树)

懒删除法



adjacency list 邻接表

Bellman-ford——按边dp找图中最短路径

n-1次松弛即可

floyd-warshall

```
def floyd():
    n = len(graph)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if graph[i][k] + graph[k][j] < graph[i][j]:
                    graph[i][j] = graph[i][k] + graph[k][j]
                    parents[i][j] = parents[k][j] # 更新父结点

# 打印路径
def print_path(i, j):
    if i != j:
        print_path(i, parents[i][j])
    print(j, end='-->')

# Data [u, v, cost]
datas = [
    [0, 1, 2],
    [0, 2, 6],
    [0, 3, 4],
    [1, 2, 3],
    [2, 0, 7],
    [2, 3, 1],
    [3, 0, 5],
    [3, 2, 12],
]

n = 4
graph = [[(lambda x: 0 if x[0] == x[1] else float('inf'))([i, j]) for j in range(n)] for i in range(n)]
parents = [[i] * n for i in range(n)] # 关键地方, i-->j 的父结点初始化都为i
for u, v, c in datas:
    graph[u][v] = c # 无向当双向

print('\nPath:')
for i in range(n):
    for j in range(n):
        print('Path({}-->{}): '.format(i, j), end='')
        print_path(i, j)
        print(' cost:', graph[i][j])
```

拓扑排序 (Topological Sorting)

在图论中, 拓扑排序 (Topological Sorting) 是一个有向无环图 (DAG, Directed Acyclic Graph) 的所有顶点的线性序列。且该序列必须满足下面两个条件:

每个顶点出现且只出现一次。

若存在一条从顶点 A 到顶点 B 的路径, 那么在序列中顶点 A 出现在顶点 B 的前面。


```
graph = [[] for _ in range(n)]
indegree = [0] * n
for a, b in edges:
    graph[a].append(b)
    indegree[b] += 1

queue = deque([i for i in range(n) if indegree[i] == 0])
visited = 0
ans = []
while queue:
    u = queue.popleft()
    ans.append(u)
    visited += 1
    for v in graph[u]:
        indegree[v] -= 1
        if indegree[v] == 0:
            queue.append(v)
print(ans)
```